



UD7. VISTAS y sinónimos



Índice

Vistas.....	2
Vistas estándares.....	3
Vistas actualizables.....	3
Vistas materializadas.....	4
Sinónimos.....	7

Vistas

Para configurar los derechos que los usuarios tienen en una base de datos y que determina el modo en el que este puede usar los datos que almacenan se llevan a cabo tareas de gestión de cuentas de usuarios, gestión de privilegios y de roles.

Estos mecanismos se convierten en metodologías para asegurar los datos y restringir el acceso, y dependerán en gran medida de las políticas sobre seguridad de datos establecidas en la organización, tanto para la creación de datos y su manipulación, como para el almacenamiento, tratamiento y eliminación de ellos.

La revisión y gestión de privilegios es una tarea importante para el administrador de la base de datos. Las vistas son un método que se usa también para el control de acceso.

Una vista personalizada es aquella que crea el usuario, ya que existen otras ya creadas y gestionadas por el sistema y que pertenecen al diccionario de datos, como son las vistas estáticas y las vistas dinámicas. Una vista tiene asociada una consulta y se maneja como una tabla lógica. La vista se basa en una o más tablas o vistas, y las primeras se denominan tablas bases de la vista.

Las **vistas estándares** son aquellas que no poseen registros, sino que almacenan el texto para ejecutar la consulta de interés. De este modo, las vistas se convierten en un mecanismo de abstracción que permite crear visiones parciales de la base de datos, ya que el usuario las usa sin conocer las tablas bases que emplea ni la consulta asociada a ella, ya que la utiliza como si de una tabla se tratase.

Las vistas se sirven de tablas u otras vistas para implementar su consulta. Cuando se accede a una vista estándar, la consulta se ejecuta siempre en ese instante, atendiendo a la situación actual de la base de datos, devolviendo los registros que han cumplido el filtro. Las vistas también admiten operaciones DML que de alguna manera afectan a las tablas bases que la consulta usa. Estas vistas que permiten este tipo de operaciones se denominan **vistas actualizables**.

Otro tipo de vistas más avanzadas son aquellas que si almacenan en una tabla los registros que en el momento de su invocación cumplen el filtro de la consulta. A estas vistas se las conoce como **vistas materializadas**.

Un usuario puede crear vistas en su propio esquema siempre que disponga del privilegio CREATE VIEW. Para poder crear vistas en esquemas de otros usuarios se debe disponer del privilegio más amplio CREATE ANY VIEW. El objeto que se usa para crear una vista debe pertenecer al esquema donde se crea, o bien, disponer del privilegio EXECUTE ANY TYPE o el privilegio EXECUTE en ese objeto.

Una vista no puede contener el operador DISTINCT, cualquier cláusula CONNECT BY, GROUP BY, ORDER BY o START WITH, o una función analítica o de agregación.

Vistas estándares

Para crear una vista estándar se usa

CREATE OR REPLACE VIEW NombreVista AS SELECT...

La sintaxis completa del comando CREATE VIEW es la siguiente:

CREATE [OR REPLACE]

[FORCE | NO FORCE]

VIEW [esquema].nombreVista[(columnas)] AS Consulta

[WITH READ ONLY | WITH CHECK OPTION];

La opción **NOFORCE** permite controlar la creación de la vista solo si las tablas bases de la consulta existen y se tiene permisos para crearla. Es la opción predeterminada. La opción **FORCE** no hace este control.

La opción **Columnas** permite renombrar las columnas usadas en la consulta. Para la vista que emplea columnas renombradas, dichos nombres serán los que se deben usar cuando se quieren hacer operaciones de select sobre la vista. Si se omite, los nombres de las columnas de la vista son los mismos que los de sus tablas bases.

Para modificar una vista ya creada se usa **ALTER VIEW** y **DROP VIEW** para eliminarla.

El nombre de la vista es único para toda la base de datos. Con la opción **REPLACE** se puede reemplazar una vista que ya existe por la nueva.

Vistas actualizables

Cuando se usa la opción **WITH READ ONLY** la tabla o la vista no puede ser actualizable. Sin embargo, con la opción **WITH CHECK OPTION** se prohíbe cualquier cambio en la tabla o vista que produciría filas que no estarían incluidas en la consulta.

Supóngase que se inserta en la tabla Producto un portátil con nombre Samsung Pavilion y erróneamente se le añade el tipo S, sobremesa, en vez del tipo P, portátil:

```
INSERT INTO Producto VALUES(6,'Samsung Pavilion 166','SP16','S',21,679,895);
```

Ahora, se quiere crear una vista sobre la tabla Producto para que seleccione aquellos que son portátiles o de sobremesa. La vista sería la siguiente:

```
CREATE VIEW Ordenadores AS SELECT codProducto, nombre, Tipo FROM Producto WHERE Tipo IN ('S','P');
```

Esta vista es actualizable y se puede comprobar que permite la actualización siguiente:

```
UPDATE Ordenadores SET Tipo='C' WHERE codProducto=2;
```

Sin embargo, debería comprobar que no debe actualizar dicho valor, ya que no cumple la condición de la consulta de la vista. Para ello, se debería haber creado la vista como se indica en el siguiente ejemplo:

```
SELECT codProducto, nombre, Tipo FROM Producto WHERE Tipo IN ('S','P') WITH CHECK OPTION;
```

Podemos en Oracle actualizar, borrar e insertar datos en vistas que sean de una sola tabla como regla general siempre que luego se cumplan las restricciones de integridad referencial en ella. Por ejemplo, si la tabla tiene un campo not null que no estamos mostrando en la vista y luego queremos insertar en ella va a falla pues introducimos un null en ese campo.

Para que en una vista que resulta de un JOIN podamos actualizar los datos, estos se deben referir a la tabla con la clave primaria preservada. Se dice que una tabla tiene la PK preservada cuando también es clave en la vista.

Dado que un join es realmente la tabla del lado muchos ampliada con la del lado uno, una vista realizada con un join es actualizable si contiene la clave de la tabla del lado muchos.

Update: una vista puede actualizar sólo atributos pertenecientes a la tabla del lado muchos.

Delete: podemos borrar filas de una vista y se borrarán de la tabla del lado muchos.

Insert : podemos insertar filas en una vista y se insertarán en la tabla del lado muchos.

Para saber que atributos se pueden actualizar en una vista usamos

{USER|DBA|ALL}_UPDATABLE_COLUMNS

ejemplo:

```
SELECT COLUMN_NAME, UPDATABLE FROM USER_UPDATABLE_COLUMNS
WHERE TABLE_NAME = '<nombre de la vista>';
```

PRÁCTICA PROPUESTA

Vistas

Para la base de datos "Tienda de informática" crea una vista con los productos y un campo con el número de veces que se ha vendido ese producto.

Usando la vista selecciona los datos de los 5 productos más pedidos.

Vistas materializadas

También existen unas vistas especiales llamadas vistas materializadas. Estas vistas si contienen registros. A diferencia de las vistas "normales", una vista materializada almacena físicamente los datos resultantes de ejecutar la consulta definida en la vista. Este tipo de vistas materializadas realizan una carga inicial de los datos cuando se definen y posteriormente con una frecuencia establecida se actualizan sus datos.

```
CREATE MATERIALIZED VIEW NombreVistaMaterializada
[TABLESPACE nombre] [PARALLEL (DEGREE n)]
[BUILD [IMMEDIATE | DEFERRED]
[REFRESH {FAST | COMPLETE | FORCE | NEVER}
{ON COMMIT | ON DEMAND | [START WITH fecha] NEXT intervalo}]
[{ENABLE DISABLE} QUERY REWRITE]
AS Consulta;
```

Los parámetros especiales para configurar una vista materializada son los siguientes:
TABLESPACE: como la vista crea un objeto físico, esta opción permite indicar el espacio de tablas en el que se creará la vista materializada.

BUILD

Establecemos la forma de carga de datos inicial de la vista

IMMEDIATE: se ejecuta la consulta en el mismo instante que se crea y la tabla se llena de los datos procedentes.

DEFERRED: la tabla no se llena hasta que la vista no se actualice o refresque.

REFRESH

Establecemos el método y la frecuencia con la que se referescan los datos.

COMPLETE la tabla se vacía, y se llena de nuevo

FAST Es un tipo de refresco incremental, sólo se refrescarán aquellos datos que se modificaron desde el último refresco. Esto es más rápido.

Para poder usar FAST es necesario configurar los archivos logs de la vista materializada, con el comando CREATE MATERIALIZED VIEW LOG.

FORCE: le indica a Oracle que si la vista puede ser refrescada con FAST lo haga, y si no puede, que use COMPLETE.

NEVER: no se efectúan refrescos de la vista materializada.

Una vez visto el modo de refresco, podemos decirle a Oracle el momento en que queremos que se realice:

ON DEMAND: Es el refresco **manual** mediante el paquete de PL/SQL DBMS_MVIEW. Podemos forzar a realizar un refresco usando para eso la función REFRESH.

DBMS_MVIEW.REFRESH ('nombre_vista');

Con la función REFRESH_DEPENDENT refrescarás todas las vistas materializadas que tengan algunas de sus "tablas base" en la lista de tablas pasada como parámetro de entrada.

DBMS_MVIEW.REFRESH_DEPENDENT ('tabla1, tabla2, tabla3, ... , tablaN');

Con la función REFRESH_ALL_MVIEWS se refrescarán todas las vistas materializadas de nuestra base de datos.

ON COMMIT: Refresco automático. La vista se refresca cada vez que se hace una operación COMMIT sobre algunas de las tablas base.

START WITH... NEXT: Otro tipo de refresco automático es el programado, en el que podemos definir el momento exacto en que queremos que se refresque nuestra vista. Para eso tenemos que definir la fecha del refresco en formato datetime y el intervalo de este. Ejemplo : REFRESH START WITH SYSDATE + 1 NEXT sysdate+7 refrescaría cada 7 días comenzando en mañana.

ENABLE QUERY REWRITE: permite indicar si Oracle puede reescribir los comandos SQL para que use la vista materializada en vez de las tablas bases asociadas a la consulta.

Para configurar el tipo de refresco de una vista materializada hay que analizar la complejidad de la vista, así como los tamaños de las tablas bases, ya que un refresco excesivamente exigente puede disminuir considerablemente el rendimiento de su uso.

La siguiente vista materializada tiene un refresco completo en el momento de su creación, debido a las opciones BUILD IMMEDIATE y REFRESH COMPLETE, y se refresca automáticamente cada día, al usar NEXT SYSDATE+1.

```
CREATE MATERIALIZED VIEW pedidoDiario
BUILD IMMEDIATE
REFRESH COMPLETE NEXT SYSDATE+1
AS SELECT * FROM Producto WHERE Actual<Limitepedidoauto;
```

Para ejecutar actualizaciones incrementales o rápidas con la opción FAST, Oracle necesita apuntar cada una de las operaciones que se han efectuado sobre las tablas bases de una vista materializada; de lo contrario, toda actualización se tendría que hacer de modo completo. Para parametrizar estos apuntes se usa el comando CREATE MATERIALIZED VIEW LOG para cada una de las tablas bases antes de crear la vista

CREATE MATERIALIZED VIEW LOG ON tablaBase WITH ROWID, PRIMARY KEY INCLUDING NEW VALUES;

No todas las vistas materializadas pueden soportar el refresco FAST. Por ejemplo las que usan funciones de agregación o las build deferred no soportan el refresco FAST

PRÁCTICA PROPUESTA

Vistas Materializadas

Creamos esta tabla:

```
CREATE TABLE tabla (campo1 int PRIMARY KEY, campo2 int);
```

```
insert into tabla values (1,2);
```

```
insert into tabla values (2,298);
```

```
insert into tabla values (3,223);
```

```
insert into tabla values (4,121);
```

```
insert into tabla values (5,34);
```

```
insert into tabla values (6,34);
```

```
insert into tabla values (7,78);
```

```
insert into tabla values (8,44);
```

```
insert into tabla values (9,34);
```

```
insert into tabla values (10,12);
```

Creamos un log para la tabla ya que vamos a hacer una vista materializada que se refresque FAST:

```
CREATE MATERIALIZED VIEW LOG ON tabla WITH PRIMARY KEY INCLUDING NEW VALUES;
```

Creamos la vista materializada que importará los datos incrementalmente cuando se haga commit:

```
CREATE MATERIALIZED VIEW vm_tabla BUILD IMMEDIATE REFRESH FAST ON COMMIT AS SELECT
* FROM tabla;
```

Comprueba que cuando se inserta un dato se refresca también en la vista materializada (mira antes y después de hacer el commit)

Crea una vista materializada con refresco manual escribe como hiciste

Haz unos inserts

Refresca la vista y mira que efectivamente están los datos.

Sinónimos

Un sinónimo es un nombre alternativo para una tabla, vista, secuencia, programa, paquete, secuencia u otro sinónimo. Esto provee transparencia e independencia sobre el propietario y el nombre real del objeto. Sin embargo, los sinónimos no son un sustituto para los privilegios de los objetos de la base de datos. Se deben conceder al usuario que usa un objeto los privilegios necesarios para el uso del sinónimo.

Un sinónimo es público si lo conocen todos los usuarios, de lo contrario se trata de un sinónimo privado. Para crear un sinónimo privado en el mismo esquema se debe poseer el privilegio CREATE SYNONYM, en diferente esquema el privilegio CREATE ANY SYNONYM y CREATE PUBLIC SYNONYM para un sinónimo público.

Para crear un sinónimo se usa el comando CREATE OR REPLACE SYNONYM con la sintaxis siguiente:

```
CREATE [OR REPLACE] [PUBLIC] SYNONYM  
[esquema.]Nombre FOR [esquema.]Objeto[@enlaceDB];
```